Sanjaq

Product Design Case Study

Turning Design Decisions into a Traceable Process.

A centralized workspace that brings project scope, design feedback, decisions, and client approvals into one structured workflow.

From scattered feedback to structured design decisions.



The project list: a live summary sentence, tiles for what needs action, and the decision chain for every project. The interface is Persian and right-to-left.

Sanjaq is a decision log for design studios. It makes every important project decision traceable: what was decided, why, by whom, on which design version, and what exactly the client approved.

ROLE	PLATFORM	LANGUAGE	PROJECT TYPE	STATUS
Product Designer / UI/UX Designer	Responsive web application	Persian (RTL)	Product design · interactive prototype	Interactive prototype with mock data

Live prototype: dibarahbar.github.io/-Sanjaq

Decisions get lost between emails, chats and files.

A design project produces hundreds of small decisions. They end up scattered across emails, chat messages and design files, so the same four questions keep coming back.

01

Which design version is final?

02

Who made this decision?

03

Where is the approval?

04

Was this change part of the original scope?

What fragmentation costs

- A verbal yes or a chat reaction is not a record.
- Changes made after approval leave no trail.
- The client cannot tell exactly what they agreed to.
- When a dispute starts, the team searches old messages for evidence.

Sanjaq is neither a design tool nor a task manager. It is a decision-management layer.

It records what was decided, why, by whom, and on which version.

How I framed the problem

This framing comes from a product brief and general studio practice. I did not run interviews or usability sessions, so I treat these needs as design hypotheses that future research should validate.

Four goals, translated into design principles.

I turned each goal into a principle I could test every screen against.

GOAL 01

Every piece of feedback and every decision is linked to a specific design version.

DESIGN PRINCIPLE

Everything attaches to a version.

Pins and feedback carry a version ID. Published versions are immutable.

GOAL 02

Client approval is explicit, verifiable and documented with a receipt.

DESIGN PRINCIPLE

Approval is a deliberate act with proof.

A two-step confirmation, then a receipt with approver, version, scope, time and fingerprint.

GOAL 03

Every post-approval change has its own traceable workflow.

DESIGN PRINCIPLE

Agreements are never silently edited.

Change requests are tied to a baseline and receipt. Acceptance creates a new baseline.

GOAL 04

AI suggestions never replace human judgment.

DESIGN PRINCIPLE

AI suggests, evidence supports, a human decides.

Drafts only, each with evidence from the brief. Missing information stays “unknown”.

Principles that cut across every screen

Rules live in the logic, not only in buttons

Locks, roles and duplicate-submit protection are enforced in the reducer, so another UI path cannot bypass them.

A clear client boundary

Client pages read from a dedicated projection. Internal notes, hours and classification never reach them, and an automated test checks this.

Honest about mock data

All data is fictional and there is no backend. The interface and the documentation say so.

The decision chain

Every project follows one chain. A link closes only when the previous link has closed, and each link points to a specific version or decision.

Brief → Scope → Version & Feedback → Decision → Client Approval → Receipt → Change Request



Project overview: the banner states the next action, and the chain appears as a timeline. Both are derived from the project's data.

Next action is derived

The banner names the next step and offers one primary action. No separate status is stored, so it cannot drift from the data.

The chain is always visible

Each row in the project list shows the same chain as a timeline, so blocked projects stand out at a glance.

Why a chain

It encodes the rule that makes decisions trustworthy: nothing is approved before its inputs are settled.

AI-assisted brief and scope: suggestions need evidence.



The scope draft: every suggestion shows its evidence, and the critical ambiguity beside the form blocks approval until it is resolved.

THE PROBLEM

A client's brief is rarely complete. If AI fills the gaps without showing where its claims come from, the team either over-trusts it or ignores it.

THE DECISION

- AI extracts a suggested goal, audience, deliverables, exclusions and acceptance criteria.
- Each item is tagged "AI suggestion" and shows a sentence quoted from the original brief.
- Anything the brief does not say stays "unknown" and is listed under open questions.
- A critical ambiguity, such as "what are the acceptance criteria?", blocks approval until the client answers or the team records an assumption.
- A person approves the scope. Only then is a locked Baseline v1 created.

WHY

An evidence link and an "unconfirmed" status keep review fast and keep responsibility with a person. Trust comes from being able to check.

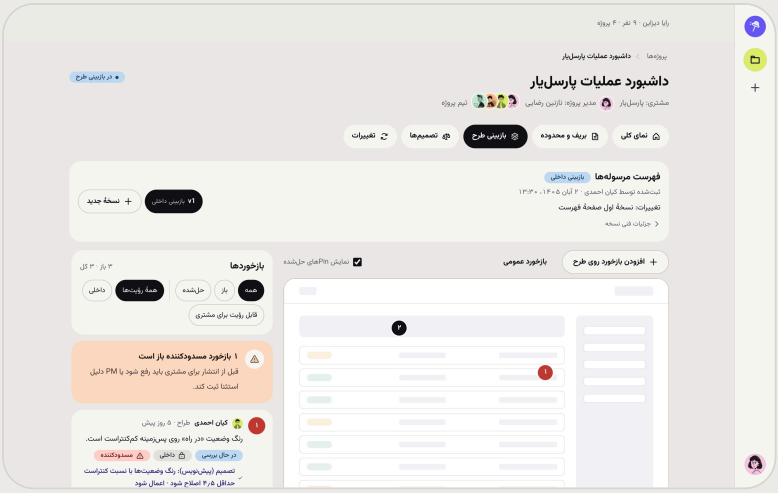
RESULTING EXPERIENCE

The reviewer sees a readiness checklist (4 of 6 complete here) and exactly what stands between the draft and approval. In the prototype the AI analysis is simulated on sample briefs; for any other brief it produces nothing rather than invented output.

B. Version-based design review

Feedback loses meaning when it is detached from the design it refers to.

- Every pin and comment is tied to one version.
- Pins use normalized coordinates (0 to 1), so they stay anchored when the image is resized.
- A published version is immutable. A correction means a new version.
- Internal and client-visible feedback are separate, with a filter that previews what the client will see.
- Blocking feedback raises a warning and prevents publication.



Design review: numbered pins, the feedback filter and the blocking-feedback banner.

C. Decision ledger

A decision that can be quietly edited cannot be trusted.

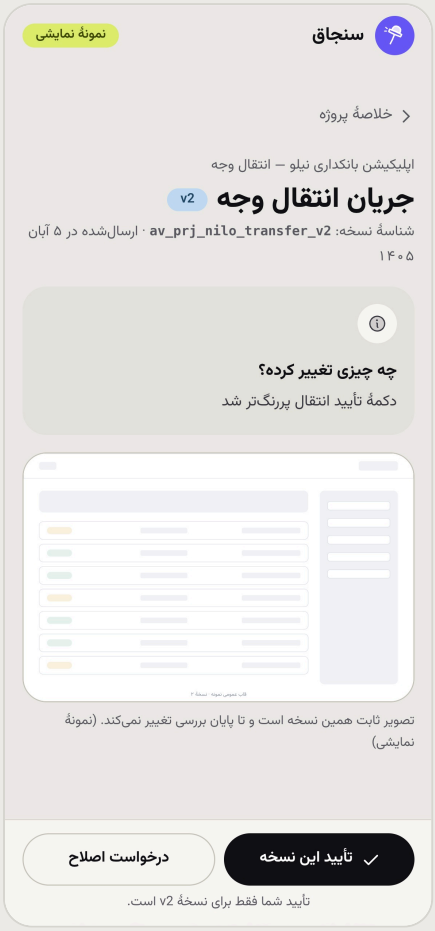
- Each decision records its rationale, who recorded it, who finalised it and which version it belongs to.
- A decision is never edited. A new decision supersedes the previous one and the link between them is preserved.

“Why did this change?” always has an answer.



Decision ledger: the reason, the recorder, the finaliser and the related version (v1, applied in v2).

Two-step client approval, with a receipt.



1 · Review

The client opens a dedicated link and sees the published version. No studio login.



2 · Confirm

The dialog states exactly what is approved. The client must tick an explicit checkbox.



3 · Receipt

A receipt records approver, version, agreed scope, time and fingerprint.

DESIGN DECISIONS

- Approval is never one tap. The dialog names the deliverable, version and ID first.
- The receipt is created once, only in the reducer. Two quick clicks never produce a second receipt.
- Receipt fields: approver, approved version, agreed scope, time (Tehran) and version fingerprint.

BOUNDARIES

- Client pages read from one projection. Internal notes, internal hours and classification never reach them, and a test verifies this.
- The dialog says plainly that this prototype approval is recorded in Sanjaq and is not a legally binding contract signature.

WHAT THE RECEIPT RECORDS

Approver · Approved version · Agreed scope (Baseline) · Time (Tehran) · Version fingerprint · Receipt ID, created once

The approval flow on mobile. A “demo sample” badge appears on every client page.

Changes after approval never touch the baseline.

Late changes are the hardest part of the problem. A change request can only start after a receipt exists, and it stays linked to the baseline and receipt it refers to.

Client request → Questions & answers → Classification → Assessment → Proposal → Decision

If the proposal is rejected

The baseline and receipt remain exactly as they were.

If it is accepted

A new Baseline v2 is created. The previous baseline is never revised directly.

A sent proposal

A proposal version cannot change after it is sent.

The result is that “was this in scope?” is answered from data, not from memory.



Before approval, “Submit change request” is disabled and the reason is written out.



After the receipt, a request can be submitted.



The request is logged, and the page states that the previous baseline and receipt are untouched.

Rules first, then screens.

The work moved through five stages. I stopped after each milestone for the product owner's approval before continuing.

1

Domain logic first

What I did: Modelled the domain and wrote a pure, tested reducer before any screen existed.

What I decided: Guards (baseline lock, roles, idempotency) live in the reducer, not only in the UI.

2

Medium-fidelity prototype

What I did: Built four complete workflows with mock data: brief to baseline, version and pins to decision, approval to receipt, change request to outcome.

What I decided: Everything that is mocked is labelled as mock.

3

High-fidelity design and system

What I did: Defined tokens, a type scale and a component library, replacing arbitrary sizes.

What I decided: Colour, type, radius and motion come only from tokens.

4

Visual direction

What I did: Refined the direction using five reference images and the product owner's feedback; two explorations were merged.

What I decided: Cream canvas, pastel tiles, black pill buttons, the chain as a timeline.

5

QA, accessibility and iteration

What I did: Ran axe-core, measured touch targets, executed all four flows by hand and audited the interface.

What I decided: Log 16 issues and fix them by severity. Most are resolved.

Why rules before the interface

A product that sells proof cannot have a lock that is only a disabled button. Any side path would bypass it. I built the rules first and designed around them, so the interface could later be redesigned from the ground up without a single rule breaking.

Honest note: No user interviews or usability sessions were conducted. Decisions were made against the product brief, the product owner's feedback and audit findings.

A calm, expressive system built from tokens.

The first pass was neutral, and the chain, the product's core value, had no visual presence. After reviewing five reference images with the product owner I moved to a cream canvas, semantic pastel tiles, black pill actions and a purple brand accent used only for focus and the logo.

Canvas
Page background
#ebeae6

Surface
Cards and tiles
#f8f7f4

Ink
Text and primary action
#101015

Brand
Focus ring and logo
#6858f5

Lime
Needs action
#dcec6c

Lavender
AI, or waiting for client
#cdc6f6

Pink
Client, or change
#f6c6e2

Sky
Version and review
#bfdaf2

Peach
Warning, or feedback
#ffbd8f

Mint
Approved
#c4ead3

Coral
Error, or blocking
#ff8f86

Colour is never the only signal. Every status also carries a text label, and focus uses a visible 2 px ring.

TYPOGRAPHY

Vazirmatn Variable with a tokenised, seven-step scale from 12 to 36 px and generous line height (1.4 to 1.85) for Persian. Hierarchy comes from size and weight.

COMPONENTS

- Button (black primary)
- Badge with status dot
- Avatar
- Segmented control
- Stage track (the chain)
- Card and tile
- Native dialog
- Form fields
- Skeleton
- Empty state



The type scale on the design-system page: Display 36 to Label 12, plus a numeric style.



The design-system sample page (development only): colour tokens with contrast ratios, type scale and components in one place.

Graphics are driven by real data (no invented percentages). Illustrations come from one set with transparent backgrounds and never sit under text. Light theme only: the dark theme was removed on purpose.

Designed right-to-left from the start, not translated.

Persian is the product's only language, so I treated RTL as the architecture rather than a mirrored layout.

RTL-first layout
Reading order, alignment and icons start from the right.

Logical CSS properties
Start and end replace fixed left and right, so nothing is hard-coded to one side.

Latin identifiers
Tokens like v1 and RC-3441E5-01 are isolated so punctuation never jumps around them.

Persian numerals
Counts, dates and money use Persian digits throughout.

Jalali calendar and Tehran time
Dates follow the Persian calendar; timestamps state "Tehran time".

Explicit currency
Amounts always carry the unit "Toman".

In the actual interface



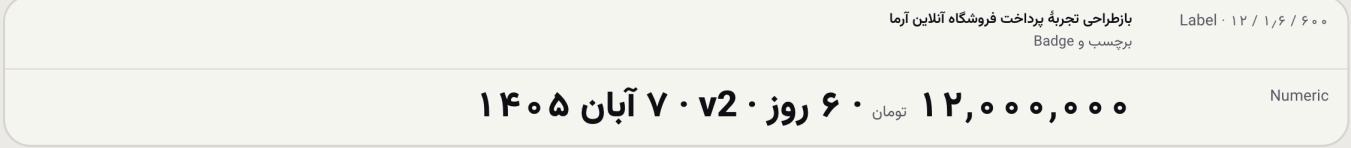
Greeting header: Persian date, digits and a summary sentence that mixes a number and text.



Approval receipt: the Latin token "Baseline v1", a Jalali date with "Tehran time", and a hexadecimal fingerprint, all inside right-to-left text.



Decision detail: Jalali dates with times, and "v1, applied in v2" inside right-to-left text.



Numeric style from the design system: amount in Toman, duration in days, version ID and Jalali date in one line.

One interface across 390, 834 and 1440 px.

The same project list at three widths. I tested the layout at each of them and kept primary controls at least 44 px tall for touch.



1440 px · Desktop



834 px · Tablet



390 px · Mobile

Desktop

A slim icon rail and a breadcrumb handle navigation. Four stat tiles sit in one row.

Tablet

Navigation moves into pill tabs at the top. Stat tiles sit in a 2×2 grid and action cards in two columns.

Mobile

A narrow single-column flow for cards and rows; stat tiles stay 2×2. Pill tabs scroll horizontally and the active tab scrolls into view.

Primary controls keep at least a 44 px touch height; only small buttons drop to 40 px.

What I verified, and what I did not.

80

automated tests passing

Scope logic, guards, versions and pins, decisions, receipts, the client-data boundary, and a full four-flow scenario.

0

axe-core violations

WCAG 2 A/AA and best practice at 390, 834 and 1440 px (15 desktop, 8 tablet and 11 mobile routes in the latest run).

0

horizontal overflow

Measured on the same routes and widths.

4

flows run by hand

Brief to baseline, version to publish, approval to receipt, change to accept and reject.

16

audit issues logged

Initial audit: technical 14/20, UX 26/40. Most are resolved; the rest are in the audit report.

Note: These are engineering checks on a prototype. They are not measures of user satisfaction or product performance.

Examples of audit fixes

- Destructive demo action had no confirmation → confirmation dialog with an explicit consequence.
- Sign-in offered 45 choices before the main task → roles and avatars simplified.
- Version ID and checksum led the review page → collapsible technical details.
- Touch targets were 40 px → at least 44 px for primary controls.
- A 707 KB bundle → split by route (entry 273 KB plus shared 291 KB).

Limitations

- Mock data only, no production backend.
- Demonstration sign-in: passwords are never checked or stored.
- No real-user usability testing.
- No real screen-reader testing yet.
- Real devices and other browsers not yet validated.
- Concurrent editing and offline mode are not implemented.

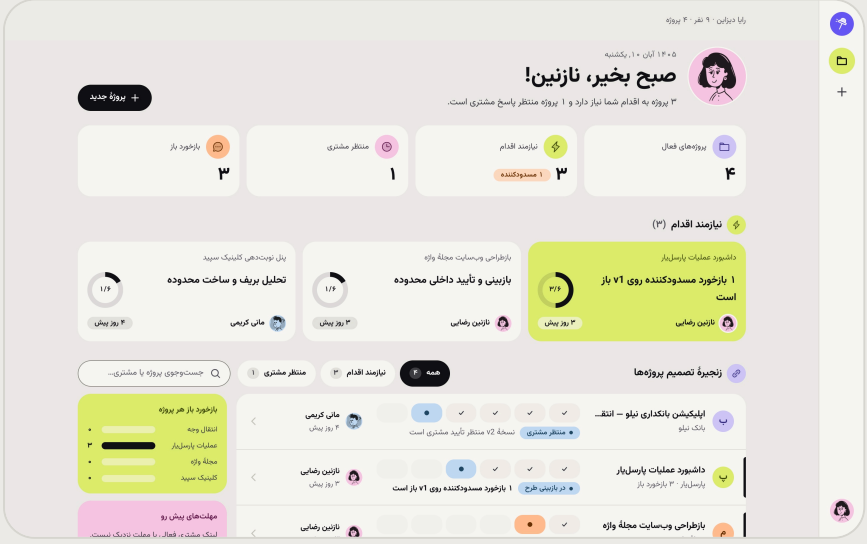
Future improvements

- Usability testing with real design studios, measuring how long it takes to find a past decision.
- A production backend and real authentication.
- Real-device and screen-reader testing.
- An expanded client workspace.

Challenges and what they taught me.

LESSON	CHALLENGE	RESOLUTION
Business rules belong in domain logic	A lock that is only a disabled button can be bypassed.	Guards moved into the reducer; the UI follows them.
Trust comes from transparency	AI output nobody can check is either over-trusted or ignored.	Evidence, “unknown” and draft status make review fast.
The core value must be visible	The first pass looked neutral; the chain was invisible.	The chain became a timeline in every row, with meaningful colour.
Tokens keep a system consistent	Arbitrary sizes made every page feel equally loud.	One token scale, applied across about 33 files.
Keep internal and client information apart	Client pages could have leaked internal notes.	A dedicated projection plus an automated test.

The most important lesson from Sanjaq was that trust is not created by the interface alone. It comes from the rules, relationships, and decisions that the interface makes visible.



Explore the live prototype

dibarahbar.github.io/-Sanjaq

Illustrations are from Genillu; the usage licence should be confirmed before public release. Vazirmatn is used under the OFL licence. All company and people names in the product are fictional.